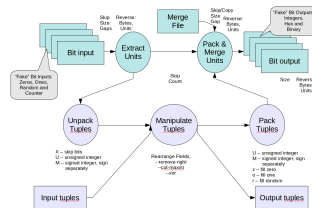


bdd — Bit Data Dump

High-performance CLI tool and Rust library to interpret, manipulate, stream, merge, and pack arbitrary-width bitstreams.

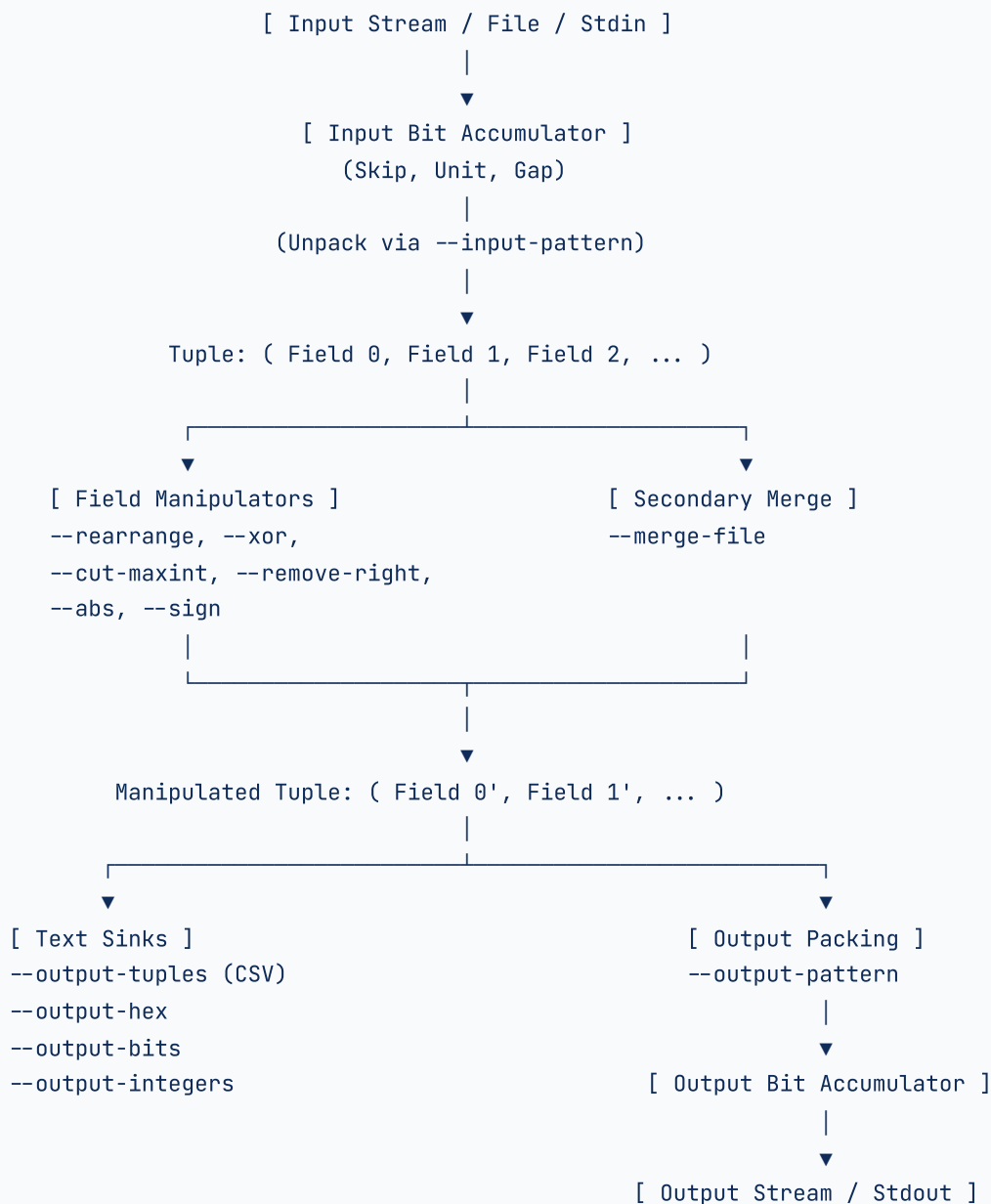
Originally created by Esa Turtiainen in Python 2 (2010), `bdd` was re-engineered in modern Rust for memory safety, 20-year maintainability, and multi-gigabit throughput with zero external C dependencies.



What is bdd?

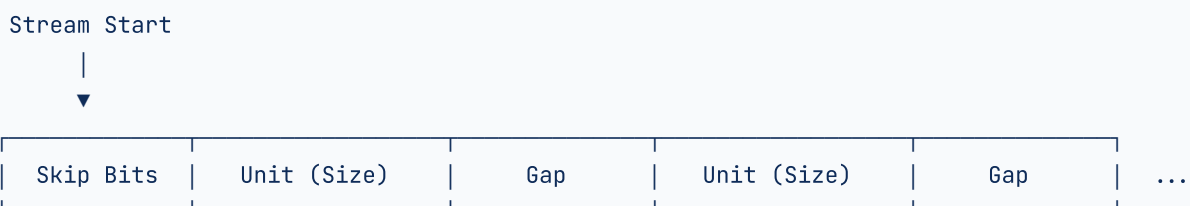
Most Unix tools (`dd`, `hexdump`, `od`, standard shell pipes) operate strictly on byte (8-bit) or machine-word (16, 32, 64-bit) boundaries. Real-world binary streams—such as embedded sensor telemetry, cryptographic protocols, SDR/radio frames, network headers, compressed payloads, and legacy archives—frequently pack unaligned fields of arbitrary bit lengths (e.g., 1-bit flags, 3-bit opcodes, 12-bit ADC samples, or 57-bit mantissas).

`bdd` is a **binary-dd for bitstreams**: it allows you to slice, unpack, inspect, transform, interleave, and repack streams of any bit width with single-bit precision.



1. Anatomy of a Bit Stream

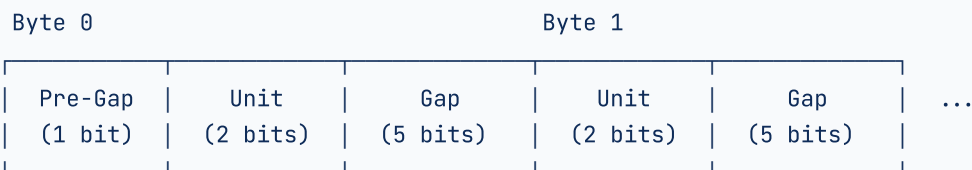
In `bdd`, all stream dimensions are measured strictly in **bits**, not bytes. A stream is modeled as a repeating series of **units**:



- `--input-unit=BITS ( -u )`: The number of bits in each repeating processing unit.
- `--input-skip-bits=BITS` : Initial offset in bits skipped before processing the first unit.
- `--input-gap=BITS` : Number of bits skipped *between* successive units.
- `--input-pregap=BITS` : Initial gap before the first unit. A convenient shorthand identical to setting `--input-skip-bits` and `--input-gap` together.
- `--input-assert-aligned` : Aborts with an error if the input stream terminates unaligned to a byte boundary.

Slicing Bits from Byte Streams

Consider extracting specific sub-fields from a continuous byte stream:



```
bdd --input-pregap=1 --input-unit=2 --input-gap=5 --output-hex < input.bin
```

Common Slicing Recipes

- **Extract the Most Significant Bit of every byte** (expands 1 bit to an 8-bit byte `0x00` or `0x01`):

```
bdd --input-unit=1 --input-gap=7 --output-unit=8 < input.bin
```

- **Pack the Least Significant Bit of bytes into a dense bitstream** (e.g., 9 bytes with LSB=1 become two bytes `0xFF` and `0x80`):

```
bdd --input-pregap=7 --input-unit=1 --output-unit=1 < input.bin
```

Unit Sizing and Alignment Rules

- **Default Unit Size**: By default, input unit size is 8 bits and output unit size is 8 bits (`bdd < foo > bar` copies bytes unchanged).
- **Expansion (Output > Input)**: If the output unit is larger than the input unit, the value is treated as an integer and the left (most significant) bits are padded with zeros.
- **Truncation (Output < Input)**: If the output unit is smaller than the input unit, the left (most significant) bits are truncated.
- **Stream Termination**: If the total bits written to an output byte stream are not a multiple of 8, the missing least-significant bits of the final byte are padded with zeros.

2. The Tuple Concept & Pattern Grammar

Rather than treating a bit unit as an opaque integer, `bdd` allows dividing a unit into named fields called a **Tuple** (analogous to a structured row in a relational database):

Raw Unit: 0 0 1 1 1 0 1 1 1 (9 bits)

Pattern: 3U 1x 2u 3M

(abs)]

| | | |

| | |

→ 3-bit Signed with Sign-Split → Yields TWO fields: [1 (sign), 1

| |

→ 2-bit Unsigned (Bit-Reversed) → Field value: 1

|

→ 1-bit Skip → Discarded (not in tuple)

→ 3-bit Unsigned Integer → Field value: 1

Unpacked Tuple: (1, 1, 1, 1)

Pattern Specifiers Reference

Every field in an `--input-pattern` or `--output-pattern` is specified as `<bits><type>`:

CODE	NAME	DESCRIPTION	ALLOWED BIT WIDTHS	VALID CONTEXT
nU	Unsigned Integer	Standard big-endian unsigned integer	Any positive integer	Input / Output
nu	Unsigned (Reversed)	Unsigned integer with reversed bit order	Any positive integer	Input / Output
nS	Signed Integer	Two's complement signed integer (returns negative values)	Any positive integer	Input / Output
ns	Signed (Reversed)	Two's complement with reversed bit order	Any positive integer	Input / Output
nM	Sign-Split Integer	Two's complement integer split into two fields: [sign, abs(val)]	Any positive integer	Input / Output
nm	Sign-Split (Reversed)	Sign-split integer with reversed bit order	Any positive integer	Input / Output
32F	32-bit Float	IEEE 754 single-precision float	Exactly 32 bits	Input / Output
32f	32-bit Float (Rev)	IEEE 754 single-precision float (reversed bytes)	Exactly 32 bits	Input / Output
64D	64-bit Double	IEEE 754 double-precision float	Exactly 64 bits	Input / Output
64d	64-bit Double (Rev)	IEEE 754 double-precision float (reversed bytes)	Exactly 64 bits	Input / Output
nC	Characters	Raw byte/character sequence (retains byte fidelity)	Multiples of 8 bits	Input / Output
nc	Characters (Reversed)	Character sequence with bit-reversed bytes	Multiples of 8 bits	Input / Output
nx	Skip / Discard	Discards n bits from input without placing them in the tuple	Any positive integer	Input only
nz	Fill Zeros	Inserts n constant zero bits	Any positive integer	Output only
no	Fill Ones	Inserts n constant one bits	Any positive integer	Output only

CODE	NAME	DESCRIPTION	ALLOWED BIT WIDTHS	VALID CONTEXT
<code>nr</code>	Fill Random	Inserts n pseudo-random bits	Any positive integer	Output only

Distinction Between `s` and `M`

- **S (Signed Integer)**: Directly represents negative values (e.g., `-5`). When printed via `--output-tuples`, it outputs `-5`.
- **M (Magnitude & Sign Split)**: Decodes a signed two's complement number into **two separate unsigned fields**:
 1. `sign`: `0` for positive, `1` for negative.
 2. `magnitude`: `abs(value)`. This allows arithmetic, filtering, or routing based on sign and absolute value independently.

3. In-Flight Tuple Manipulation

Once unpacked into a tuple, fields can be transformed using pipeline manipulators before repacking. Fields are referenced by index starting from `0`. Negative indices count from the end of the tuple (`-1` = last field, `-2` = second to last):

Manipulators

- `--rearrange=F0,F1,...`: Reorders, duplicates, or selects specific fields.

```
# Swap field 0 and field 1:
bdd --input-pattern=1U2U --rearrange=1,0 --output-pattern=2U1U
```

- `--cut-maxint=FIELD,MAX`: Clamps field value within `[-MAX, MAX]`.
- `--remove-right=FIELD,BITS`: Bitwise right-shifts field by `BITS`, stripping low-order bits.
- `--xor=FIELD,BITS`: Bitwise XORs field with a mask of `BITS` ones (inverts N bits).
- `--abs=FIELD`: Converts a signed field to its absolute value.
- `--sign=FIELD`: Isolates the sign bit of a signed field (`0` = positive, `1` = negative).

Advanced Pipeline Recipe

Problem from presentation: Input is a stream of signed bytes (`8M`). We want to clamp the value so `-128` becomes `-127`, isolate the magnitude, and extract only the most significant bit of the magnitude:

```
bdd --input-pattern=8M --rearrange=-1 --cut-maxint=0,127 --remove-right=0,6 --output-unit=1 <
input.bin
```

Explanation:

1. `8M` produces `[sign, magnitude]`.
2. `--rearrange=-1` selects only the last field (magnitude).
3. `--cut-maxint=0,127` clamps the magnitude to 127.
4. `--remove-right=0,6` shifts right by 6 bits, leaving the 7th bit (MSB of magnitude).
5. `--output-unit=1` outputs the single bit.

4. Synthetic ("Fake") Streams & Text Formats

`bdd` can synthesize bitstreams without requiring input files, and output in human-readable or script-friendly formats:

Synthetic Stream Sources

- `--input-zeros ( -0 )`: Endless stream of zero bits.
- `--input-ones ( -1 )`: Endless stream of one bits.
- `--input-random ( -r )`: Pseudo-random bits (PRNG).
- `--input-counter ( -c )`: Sequential integer counter starting at 0, incrementing by 1 per unit.
- `--count=N`: Limit processing to N units (0 = infinite).
- `--skip=N`: Skip N initial units before processing.

Human-Readable & Script Sinks

- `--output-hex ( -x )`: Formatted hexadecimal representation per unit, grouped neatly into power-of-two line widths.
- `--output-bits ( -b / --output-bit )`: ASCII bit strings (`'0'` and `'1'`) per unit.
- `--output-integers ( -i / --output-integer )`: One unsigned decimal integer per line (ideal for Unix pipelines: `awk`, `sort`, `uniq`).
- `--output-tuples ( -T / --output-tuple )`: Comma-separated values per unit.
- `--input-tuples ( -t )`: Ingest comma-separated values directly from stdin/file into tuples.

Packing from Text & Tuples

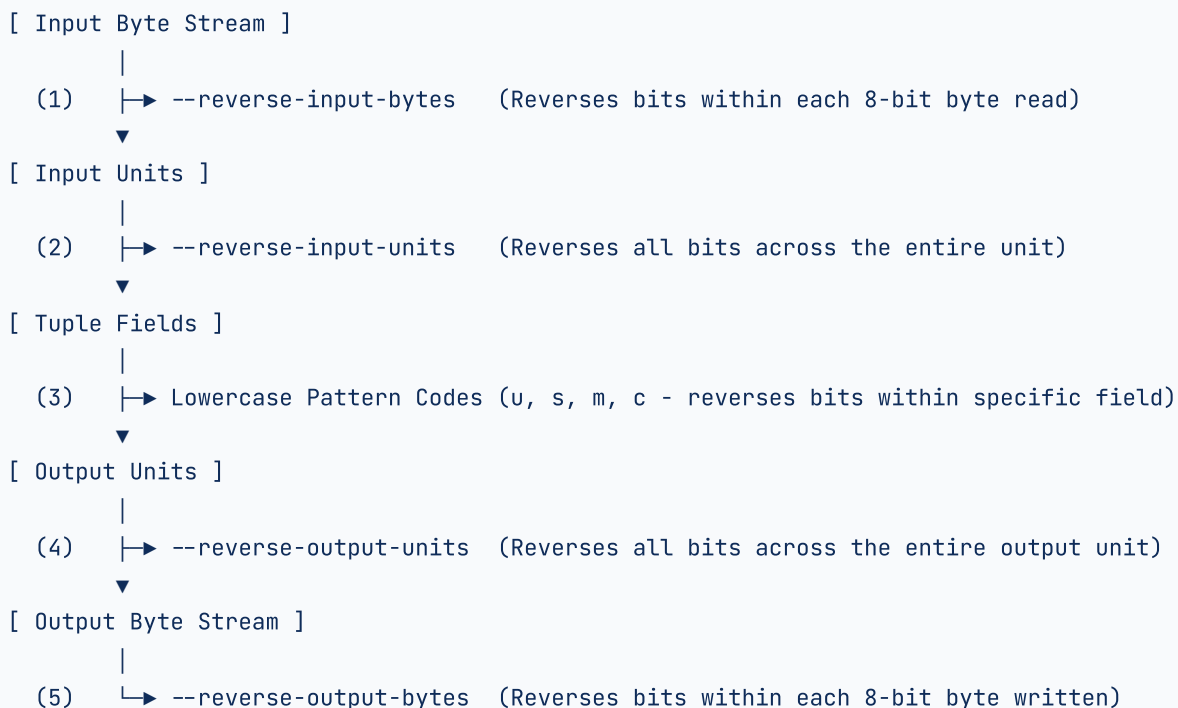
```
# Pack octal numbers from text tuples into binary:
printf "1,2,3\n3,7,7\n" | bdd --input-tuples --output-pattern='2U3U3U' | od -t o1
# Outputs: 123 377

# Pack two 4-bit numbers into hex bytes:
echo -en "1,2\n3,4" | bdd --input-tuples --output-pattern=4U4U | od -t x1
# Outputs: 12 34

# Pack text numbers into 64-bit IEEE double-precision floats:
printf "1.0\n2.0\n" | bdd --input-tuples --output-pattern=64D > doubles.bin
```

5. De-mystifying Bit Reversals

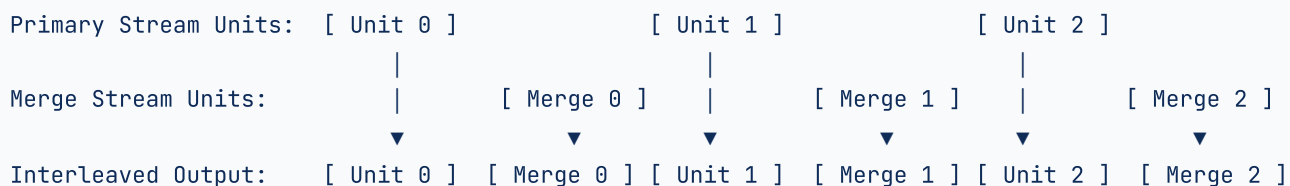
Reversing bits in binary processing can easily become confusing. `bdd` cleanly isolates bit reversals into four distinct operational layers:



- `--input-little-endian` : Shorthand combining `--reverse-input-bytes` and `--reverse-input-units`.
- `--output-little-endian` : Shorthand combining `--reverse-output-bytes` and `--reverse-output-units`.

6. Stream Merging & Interleaving

The merge stream reads a secondary file and interleaves its data into the primary stream:



- `--merge-file=PATH` : Interleave data from a secondary file (`"-"` for stdin).
- `--merge-unit=BITS` : Size of merge units in bits (default: 8).
- `--merge-copy-first=BITS` : Copies an initial bit header/preamble from the merge file *before* starting the interleaved loop.

Example: Hex Dump with Interleaved Memory Addresses

Interleave a 12-bit linear address counter with actual data bytes from `/etc/passwd` :

```
bdd --input-counter --count=16 --input-unit=12 --output-unit=12 --merge-file=/etc/passwd --  
output-hex
```

Output:

```
000 72 001 6f 002 6f 003 74 004 3a 005 78 006 3a 007 30  
008 3a 009 30 00a 3a 00b 72 00c 6f 00d 6f 00e 74 00f 3a
```

7. Command Line Options Reference

Usage: bdd [OPTIONS] [FILE]

Arguments:

[FILE] Input file (defaults to standard input '-')

Input Unit & Pattern Options:

-p, --input-pattern <PATTERN>	Bit pattern to unpack input (e.g. "3U1x2u3M")
-u, --input-unit <BITS>	Input unit size in bits (shorthand for <BITS>U)
--input-skip-bits <BITS>	Initial bit offset before first unit [default: 0]
--input-gap <BITS>	Bit gap skipped between units [default: 0]
--input-pregap <BITS>	Shorthand for initial skip & gap [default: 0]
--input-assert-aligned	Error if EOF is not byte-aligned

Synthetic Stream Sources:

-c, --input-counter	Generate sequential counter numbers (0, 1, 2...)
-0, --input-zeros	Generate endless stream of zero bits
-1, --input-ones	Generate endless stream of one bits
-r, --input-random	Generate pseudo-random bits
-t, --input-tuples	Read comma-separated tuple lines from text input
--skip <UNITS>	Skip initial N units [default: 0]
--count <COUNT>	Process at most N units (0 = infinite) [default: 0]

Bit Reversal Options:

--reverse-input-bytes	Reverse bit order within each input byte
--reverse-input-units	Reverse bit order across entire input unit
--input-little-endian	Combined byte and unit reversal for input
--reverse-output-bytes	Reverse bit order within each output byte
--reverse-output-units	Reverse bit order across entire output unit
--output-little-endian	Combined byte and unit reversal for output

Tuple Manipulators:

--rearrange <FIELDS>	Reorder output fields (e.g. "1,0" or "-1,0")
--cut-maxint <F,MAX>	Clamp field F to [-MAX, MAX]
--remove-right <F,BITS>	Right-shift field F by BITS
--xor <F,BITS>	Bitwise XOR field F with mask of BITS ones
--abs <FIELD>	Replace signed field F with abs(F)
--sign <FIELD>	Replace signed field F with sign bit (0/1)

Output Unit & Pattern Options:

--output-pattern <PATTERN>	Bit pattern to pack output
--output-unit <BITS>	Output unit size in bits (shorthand for <BITS>U)
-x, --output-hex	Output units as formatted hexadecimal strings
-b, --output-bits	Output units as ASCII bit strings ('0' and '1')
-i, --output-integers	Output unsigned integer per unit (one per line)
-T, --output-tuples	Output fields as comma-separated tuples

Merge Options:

--merge-file <PATH>	Interleave stream from secondary file
---------------------	---------------------------------------

<code>--merge-unit <BITS></code>	Bit width of each merge unit [default: 8]
<code>--merge-copy-first <BITS></code>	Copy initial header bits from merge file first
General:	
<code>-h, --help</code>	Print help
<code>-V, --version</code>	Print version

8. Performance & Throughput Benchmarks

`bdd` achieves high throughput across arbitrary bit boundaries, balancing hardware register acceleration for sub-64-bit units with arbitrary-precision arithmetic for large bignum fields.

Measured via `make bench` (`benches/throughput.rs`) on Linux x86_64:

Operation	Total Volume	Throughput (bits/s)	Throughput (bytes/s)	Notes
Hardware 64-bit Bit Reversal	128 Mbits	16.00 Gbps	2,000 MB/s	Direct CPU <code>u64::reverse_bits()</code>
Bignum 1024-bit Packing/Unpacking	10.2 Mbits	5.27 Gbps	658.6 MB/s	Large-block bignum bitfield packing
Bignum 256-bit Packing/Unpacking	12.8 Mbits	1.90 Gbps	237.2 MB/s	SHA-256 size field packing/unpacking
Synthetic 8-bit Linear Stream	16.0 Mbits	234.5 Mbps	29.3 MB/s	Continuous bit generation & sink flush
1-bit Single-Bit Resolution Stream	0.5 Mbits	43.3 Mbps	5.4 MB/s	Single-bit slice accumulation & packing
Bignum 1024-bit Bit Reversal	5.1 Mbits	39.4 Mbps	4.9 MB/s	Full arbitrary-precision bit reversal
Unaligned 3-bit to 8-bit Extraction	3.0 Mbits	39.0 Mbps	4.9 MB/s	Cross-byte boundary accumulation
Tuple Pipeline (2U3U3U → Rearrange)	2.0 Mbits	29.7 Mbps	3.7 MB/s	Multi-field unpack, reorder & repack

9. Architecture & Codebase Design

The Rust implementation is organized cleanly into modular crates:

```
src/
├─ lib.rs          # Public library crate interface
├─ main.rs         # Thin 18-line executable wrapper
├─ error.rs        # Strongly-typed BddError hierarchy
├─ counter.rs      # Unit and skip counting logic
├─ field.rs        # Arbitrary-precision Field enum & hardware-accelerated bit-reversals
├─ pattern.rs      # Grammar parser, TupleUnpacker, and TuplePacker
├─ stream.rs       # Stream generators (File, Counter, Zeros, Ones, Random, Tuples)
├─ sink.rs         # Output writers (Binary, Hex, Bit, Integer, CSV Tuples)
├─ manipulator.rs  # Field transformations (Rearrange, Xor, CutMaxint, Abs, Sign)
├─ cli.rs          # Clap CLI definition & validation rules
└─ engine.rs       # End-to-end pipeline execution orchestrator
```

Key Design Principles

1. **Strict Type Safety:** All errors flow through `BddError`. Functions return `Result<T, BddError>` instead of panicking or calling `std::process::exit`.
2. **Fast-Path Bit Reversal:** Sub-64-bit integer bit reversals execute via direct hardware `u64::reverse_bits()`, falling back to `BigUint` bit arithmetic only when necessary.
3. **Byte-Level String Integrity:** The `C` and `c` pattern types store raw bytes internally (`Field::Bytes`) rather than lossy UTF-8 conversions, guaranteeing bit-perfect roundtrips.
4. **Automated Verification:** Integrated test runner runs native Rust unit tests, bignum tests, and legacy golden-file integration tests.

10. Development, Testing & Documentation

Run the test suite:

```
make test
# or: cargo test --all-targets --all-features
```

Run benchmarks:

```
make bench
# or: cargo bench
```

Run code linter and formatting checks:

```
make lint
make check-fmt
```

Documentation & PDF Generation

All project documentation compiles into clean, print-ready vector PDF and HTML files:

- [README.pdf](#) : Generated from `README.md` via headless Chromium / Puppeteer with vector math, diagrams, and GFM styling.
- [docs/bdd.1.pdf](#) : Unix manual page rendered as clean vector PDF via `groff` and `ps2pdf` .
- [docs/bdd.1.html](#) : Unix manual page rendered as standalone HTML.
- [docs/Presentation.pdf](#) : Architectural slide deck compiled from OpenDocument Presentation (`legacy/old_src/Presentation.odp`) via headless LibreOffice.

To regenerate all documentation artifacts in one command:

```
make docs
```

License

GPL-3.0-or-later. Original author: Esa Turtiainen.